

Practica 7 - Sistemas y tipos de datos

Ejercicio 1 - Sistema de tipos

1. Que es un sistema de tipos y cual es su principal funcion?

Un sistema de tipos es un componente fundamental de un lenguaje de programacion que se encarga de asignar un tipo a los diferentes valores y expresiones dentro del programa.

La principal funcion de un sistema de tipos es verificar y garantizar la consistencia de los tipos de datos utilizados en el codigo, tanto en tiempo de compilacion como en tiempo de ejecucion. Esto ayuda a prevenir errores como intentar sumar un numero con una cadena o llamar a una funcion con un numero incorrecto de argumentos.

2. Definir y contrastar las definiciones de un sistema de tipos fuerte. Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

Tipado fuerte: Un lenguaje tiene tipado fuerte si no permite realizar operaciones entre tipos incompatibles sin conversion explicita. Los errores de tipos se detectan y se previenen.

Ejemplo:

```
a = 3 + "4" → ❌ lanza un error
```

Pero por ejemplo Java, requiere conversión explícita para mezclar tipos incompatibles, como convertir `String` a `int`.

Tipado debil: Un lenguaje tiene tipado debil si permite operaciones entre tipos incompatibles mediante conversiones implicitas, lo que puede provocar errores dificiles de detectar.

```
`"3" + 4` → `"34"` (convierte el número a cadena).
```

Resumen comparativo:

Característica	Tipado fuerte	Tipado débil
Seguridad de tipos	Alta, errores evitados	Baja, errores posibles en ejecución
Conversión implícita	No (o muy limitada)	Sí
Ejemplos	Python, Java, Haskell	JavaScript, C, Perl

3. Es posible caracterizar el tipado como estático o dinámico. ¿Qué significa esto? Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

Tipado estatico: La verificacion de tipos se realiza en tiempo de compulacion. Los errores de tipo se detectan antes de ejecutar el programa.

Un ejemplo de tipado estatico es Java, se debe declarar el tipo de cada variable.

```
int x = 30;
```

Tipado dinamico: La verificacion de tipos ocurre en tiempo de ejecucion. Los tipos no se asocian a las variables sino a los valores.

Un ejemplo de tipado dinamico es Python, las variables no tienen tipo fijo, no se declaran con un tipo, si no que su tipo puede cambiar dinamicamente.

```
x = 10  
x = "hola"
```

Ejercicio 2- Tipos de datos:

1. Definicion de tipo de datos:

Un tipo de dato es una categoria de valores que define que tipo de dato puede almacenar una variable y que operaciones se pueden realizar sobre ella. Determina el formato, el tamaño en memoria y el conjunto de operaciones validas.

2. Que es un tipo predefinido elemental?

Un tipo predefinido elemental (o primitivo) es un tipo de dato basico que ya esta incluido en el lenguaje de programacion. No necesita ser definido por el usuario y es fundamental para construir programas.

Ejemplo en Java, los tipos de datos elementales serian: int, float, char, boolean, String

3. Que es un tipo definido por el usuario?

Un tipo definido por el usuario es aquel que se crea a partir de tipos ya existentes, generalmente para representar estructuras mas complejas o personalizadas. Permiten agrupar distintos datos en una sola entidad logica.

Por ejemplo en Java, se pueden crear nuevos tipos de datos a traves de clases

```
class Persona {  
    String nombre;
```

```
int edad;
}
```

Persona es un nuevo tipo definido por el usuario, que agrupa un nombre (String) y una edad (integer)

Ejercicio 3 - Tipos compuestos:

1. Dar una breve definicion de producto cartesiano, correspondencia finita, uniones y tipos recursivos.

- **Producto cartesiano:** Es una combinacion de varios tipos de datos en una sola estructura, donde cada campo tiene un valor. Los registros o clases son ejemplos de tipo de producto cartesiano.
- **Correspondencia finita:** Es una estructura que asocia un conjunto finito de claves con valores. Tambien se conocen como diccionario, mapa o tabla hash.
- **Uniones:** Representa un tipo de dato que puede ser uno entre varios posibles tipos o valores.
- **Tipos recursivos:** Son tipos de datos que se definen en terminos de di mismos, permitiendo estructuras como arboles.

2. Clasificación de los fragmentos de código

Código	Tipo de dato compuesto	Justificación
Java <pre>class Persona { String nombre; String apellido; int edad; }</pre>	 Producto cartesiano (Product type)	Agrupar campos con diferentes tipos de datos.
C <pre>typedef struct _nodoLista {...} typedef struct _lista {...}</pre>	 Producto cartesiano  Tipo recursivo	struct agrupa campos → producto; nodoLista se refiere a sí mismo → recursivo.
C <pre>union codigo { int numero; char id; };</pre>	 Unión (Sum type)	Puede contener un solo campo activo a la vez .
Ruby <pre>hash = { uno: 1, dos: 2, ... }</pre>	 Correspondencia finita (Finite map)	Es un diccionario clave-valor.

Código	Tipo de dato compuesto	Justificación
PHP <pre>function doble(\$x) { return 2 * \$x; }</pre>	✗ Ninguno	No es un tipo de dato, sino una función.
Python <pre>tuple = ('physics', 'chemistry', 1997, 2000)</pre>	■ Producto cartesiano (Product type)	Agrupar distintos valores en una tupla.
Haskell <pre>`data ArbolBinarioInt = Nil</pre>	Nodo int ...`	■ Producto cartesiano ■ Unión (Sum type) ■ Tipo recursivo
Haskell <pre>`data Color = Rojo</pre>	Verde	Azul`

Ejercicio 4 - Mutabilidad/inmutabilidad

1. Definir mutabilidad e inmutabilidad respecto a un dato. Dar ejemplos en al menos 2 lenguajes.

Mutabilidad significa que el valor de un objeto puede cambiar después de haber sido creado. En cambio, un objeto inmutable no puede modificarse una vez creado.

Ejemplo en python:

Inmutables: `int`, `float`, `str`, `tuple`.

```
x = 5
x = x + 1 # No modifica el 5 original, crea un nuevo objeto con valor 6
```

Ejemplo en Ruby:

Por defecto, muchos objetos como strings son mutables. Sin embargo, se puede volver inmutable un objeto usando `#freeze`.

```
texto = "Hola"
texto.freeze
# texto << " mundo" # Esto lanzaría un error porque el objeto está congelado
```

2. Dado el siguiente código:

```
a = Dato.new(1)
a = Dato.new(2)
```

¿Se puede afirmar que el objeto `Dato.new(1)` es mutable?

Respuesta: No, no se puede afirmar que el objeto `Dato.new(1)` sea mutable solo con este código. Habría que ver como esta implementada la clase `Dato`.

Justificación:

El código muestra que la **variable** a **cambia su referencia**, primero apuntando a un objeto (`Dato.new(1)`) y luego a otro (`Dato.new(2)`). Pero esto **no implica que el objeto original haya sido modificado**, solo que fue reemplazado por otro.

La mutabilidad se refiere a **modificar el estado interno de un objeto**, no a cambiar qué objeto está referenciado por una variable. Para saber si `Dato.new(1)` es mutable, deberíamos ver si se puede modificar su contenido (por ejemplo, cambiar su valor interno sin crear un nuevo objeto).

Manejo de punteros:

1. Permite C tomar el L-Valor de las variables? Ejemplificar.

Si, C permite tomar el L-Valor de las variables, lo cual significa obtener la dirección de memoria donde esta almacenada una variable. Esto se logra usando el operador `&`, el cual es un operador de dirección.

```
int x = 10;
int* ptr = &x; // aquí se toma el l-valor de x
```

En este caso, `&x` obtiene la dirección de memoria de la variable `x`, y esa dirección es almacenada en el puntero `ptr`.

2. Que problemas existen en el manejo de punteros? Ejemplificar.

El manejo de punteros en C puede generar problemas si no se usan correctamente.

Por ejemplo pueden generarse errores como:

- Acceso no valido a memoria (punteros colgantes):
Ocurren cuando un puntero apunta a una dirección de memoria que ya no es valida.

```
int* ptr;
*ptr = 5; // ERROR: ptr no apunta a una dirección válida
```

- Doble liberacion de memoria:
Liberar dos veces la misma dirección de memoria puede producir comportamientos indefinidos.

```
int* p = malloc(sizeof(int));
free(p);
free(p); // ERROR: doble liberación
```

Ejercicio 6

1. Que características debe cumplir una unidad para que sea un TAD?

Un tipo abstracto de datos (TAD) es una estructura que define un tipo de dato junto con un conjunto de operaciones validas sobre ese dato, sin exponer su implementacion interna.

Para que una unidad sea considerada un TAD, debe cumplir con las siguientes características:

- Encapsulamiento: Los datos internos estan ocultos y solo pueden ser manipulados mediante operaciones definidas (metodos).
- Interfaz definida: Se especifica que operaciones estan disponibles y como se usan sin detallar como estan implementadas.

2. Dar algunos ejemplos de TAD en lenguajes tales como ADA, Java, Python, entre otros.

- ADA:
Usa el concepto de packages para definir TADs ocultando los detalles de implementacion.
 - Ejemplo: un TAD stack (pila) implementado como un paquete que exporta `Push` , `Pop` , `IsEmpty` , etc.
- Java:
Las clases son usadas como TADs
Ejemplo:

```
public class Cola {
    private LinkedList<Integer> datos = new LinkedList<>();

    public void encolar(int elemento) { datos.addLast(elemento); }
    public int desencolar() { return datos.removeFirst(); }
    public boolean estaVacia() { return datos.isEmpty(); }
}
```

- Python:
Las clases tambien son usadas como TADs en python.

```
class Cola:
    def __init__(self):
        self.elementos = []
```

```
def encolar(self, valor):
    self.elementos.append(valor) # Agrega al final de la lista

def desencolar(self):
    if not self.esta_vacia():
        return self.elementos.pop(0) # Elimina el primer elemento
    else:
        raise IndexError("La cola está vacía")

def esta_vacia(self):
    return len(self.elementos) == 0

def ver_primero(self):
    if not self.esta_vacia():
        return self.elementos[0]
    else:
        raise IndexError("La cola está vacía")
```